1. 先搞懂 IDEA 运行的全过程

1. 你写两个源码文件：`Test.java`、`Dog.java`
2. 点运行的时候，IDEA**先把这两个文件全部编译**
   - `Test.java` → `Test.class`
   - `Dog.java` → `Dog.class`
3. 然后执行 `Test` 里面的 `main` 方法
   - 代码执行到 `Dog d1 = new Dog();`
   - JVM 发现需要 `Dog` 这个类，自动去当前包加载 `Dog.class`

> 不是只编译 Test.java！IDEA 会自动把同包下所有需要用到的源码一起编译。

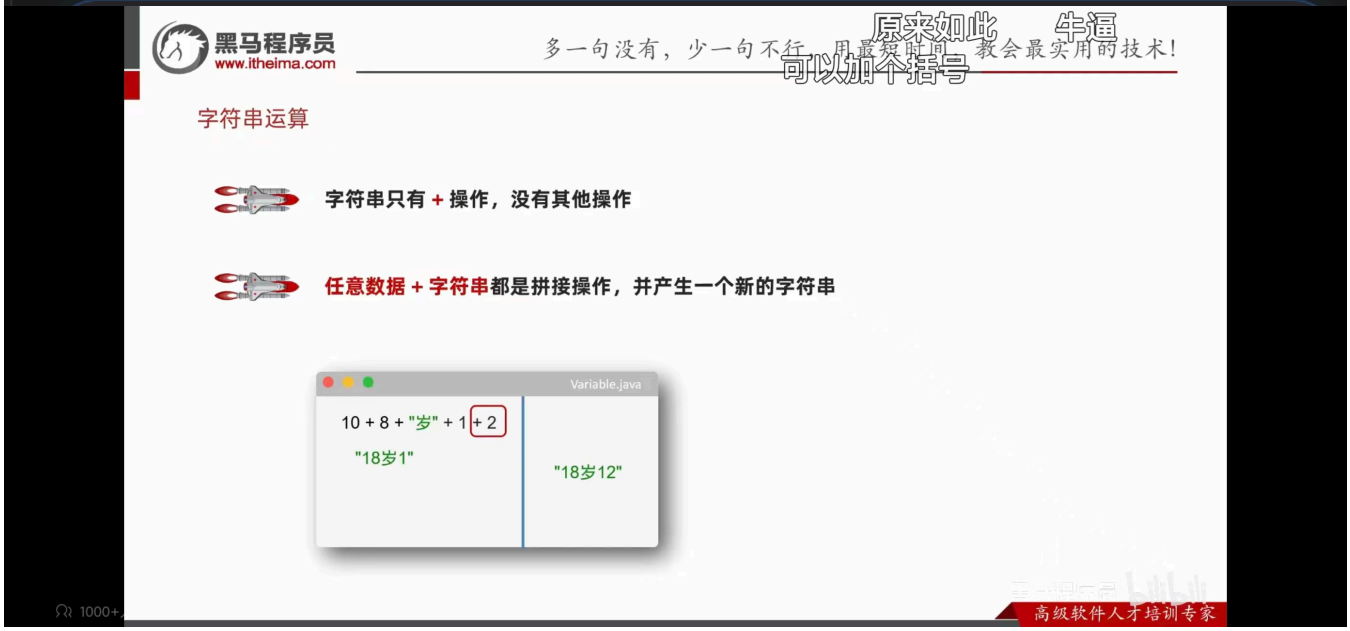
2. 关键：同包（同一个文件夹）

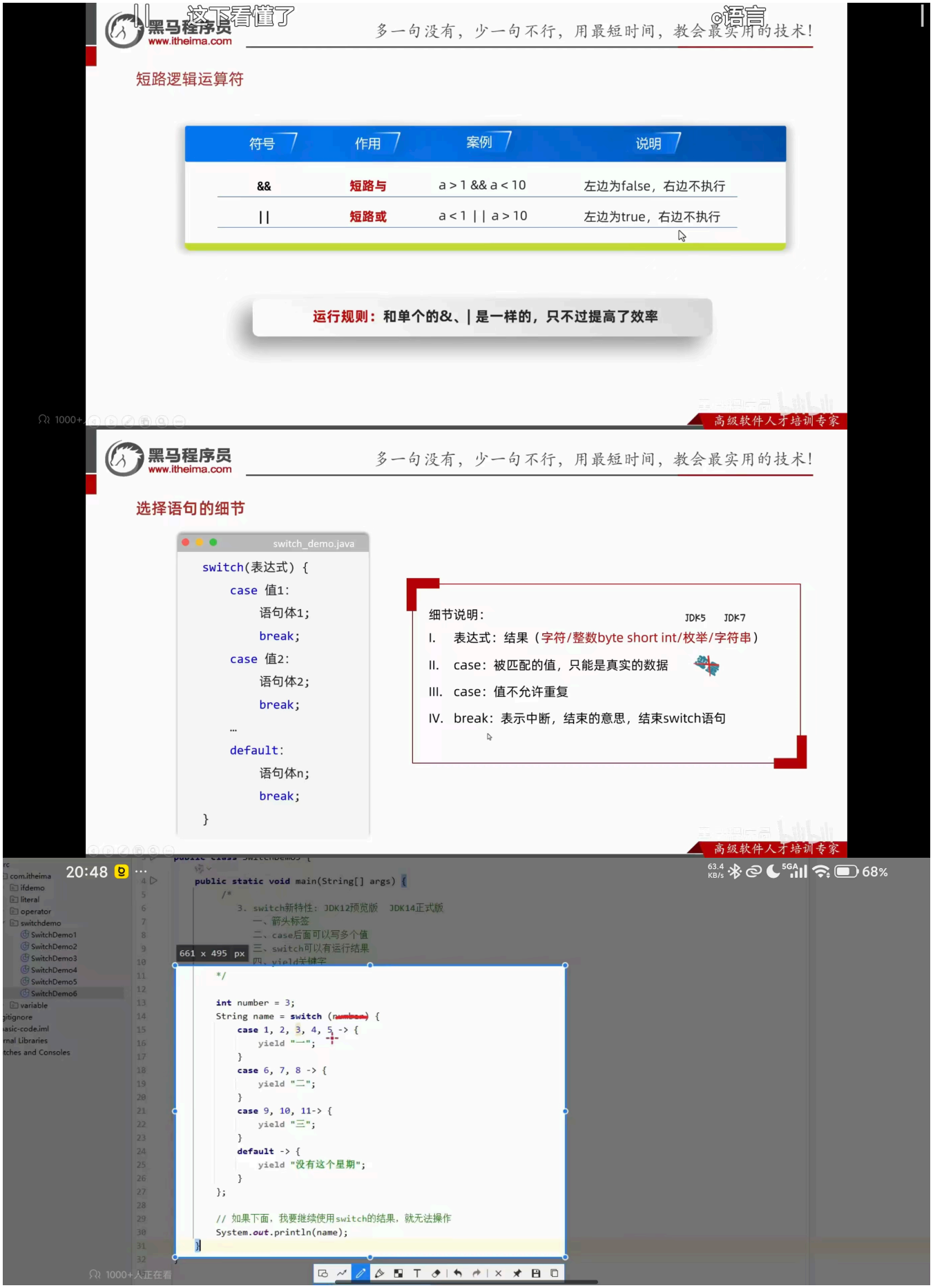
截图里两个文件都在：`com.itheima.ooptest1` 包

同包里面的类，**互相直接可见，不用 import 导入。**

所以 `Test` 写 `new Dog()`，编译器一眼就知道 Dog 是这个包里的 Dog 类。

3. 容易混淆的点





只有一句话yield和{}可以省略,如果有返回值必须}后有；就相当于赋值一个变量

AI 生成可能有误 请核实

✅ 分两种情况，一句话结论

1. **普通 switch 语句（老冒号写法，只是执行代码，不赋值）：default 可以省略**

```java
int b=10;
switch(b){
    case 1:
        System.out.println("1");
        break;
    case 10:
        System.out.println("10");
        break;
}
```

没有匹配的 case，直接跳出 switch，**不会报错。**

2. **switch 表达式（`int c = switch(...)`，就是你现在写的这种，会返回值赋值给变量）：default**** 不能随便省略**
   编译器强制要求：**所有可能输入，都必须能返回一个结果。**
   - 如果是`int`、`char`这种范围很大的类型：**必须写 default 兜底**，不能省略，就是你刚才报的错。
   - 如果是**枚举 Enum**，你把每一个枚举值全部写进 case，这时可以**不用 default**（全部情况都覆盖了）。

你的代码场景

2️⃣ while(true)

java

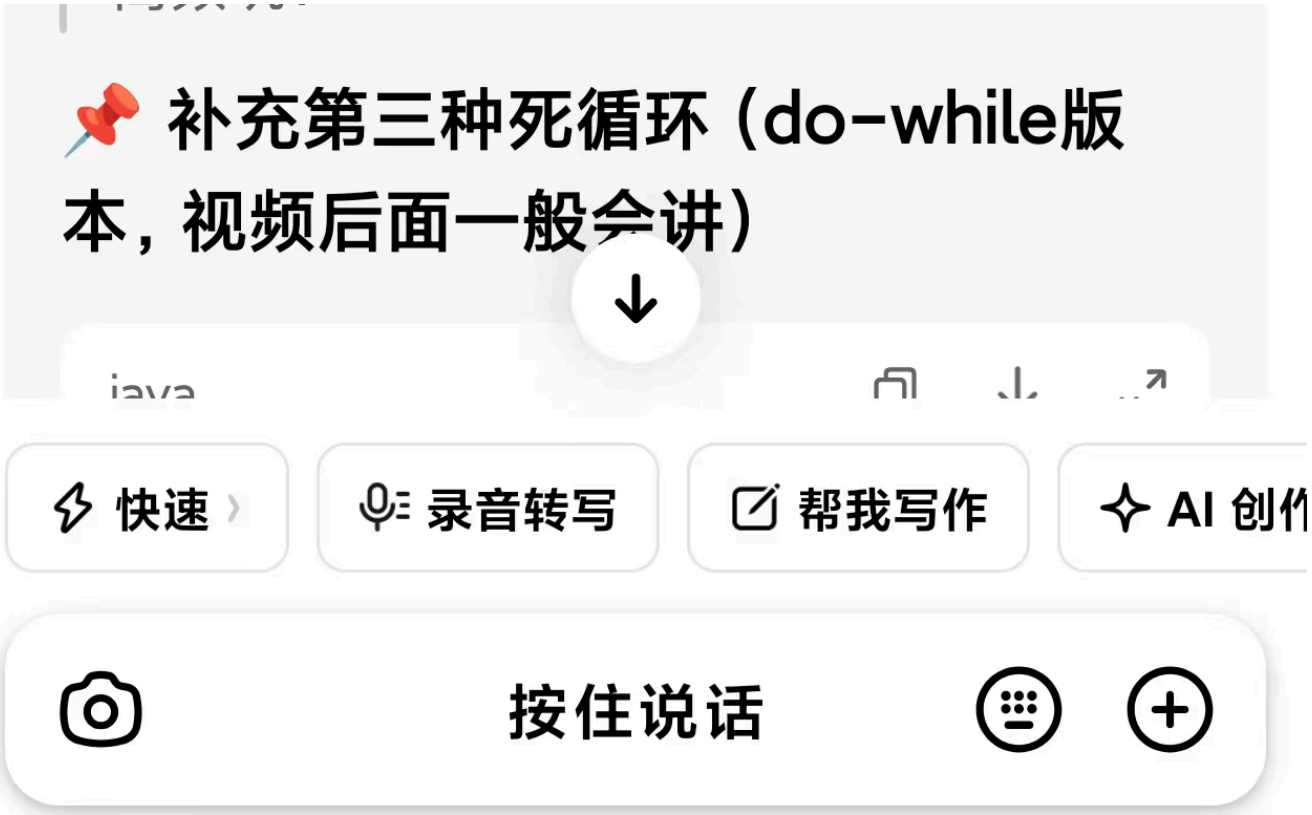
```
while (true) {
    System.out.println("阿玮哥哥好帅");
}
```

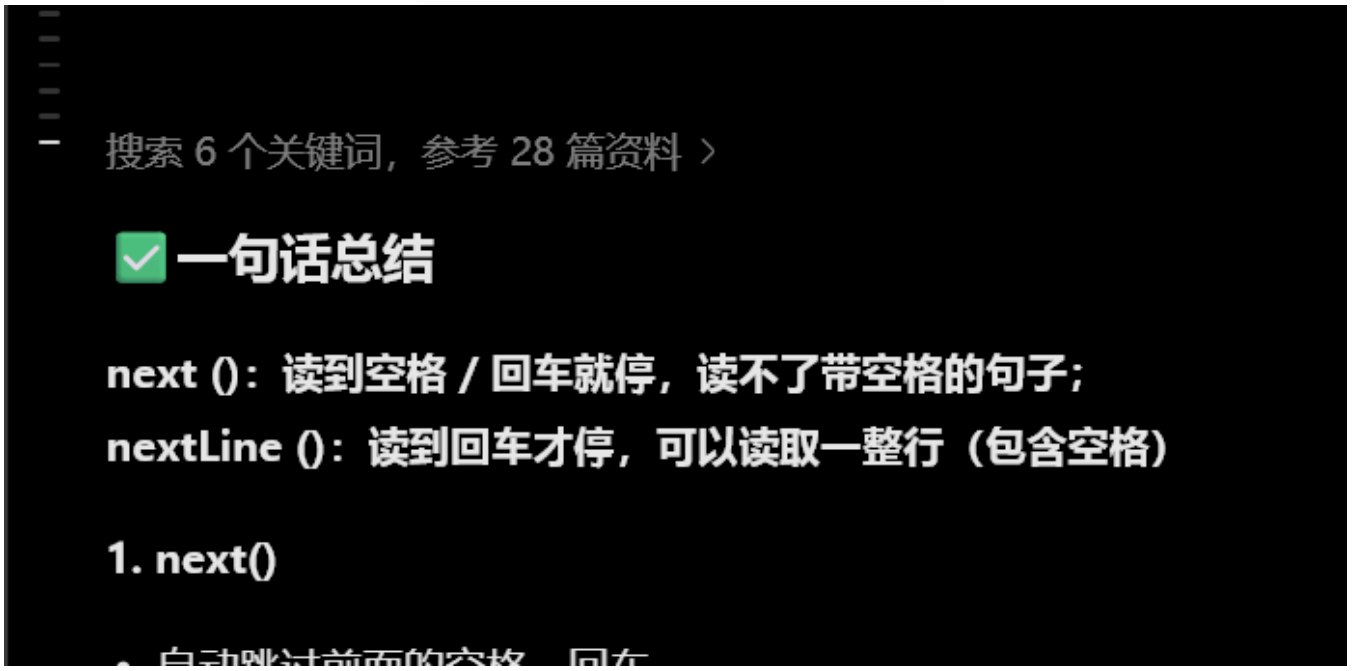
> while括号里写布尔常量 `true`，条件永远为真，循环永远不会退出 → 死循环

📌 考点重点（刚好承接你上一题！）

❗ 还记得刚才问的：**true能不能换成1?**
👉 这里 `while(true)` 里面必须是**boolean类型true，不能写 while(1)，Java编译直接报错！**

> C语言可以写while(1)，Java不行，这就是高频坑！





System.out.print()括号里面不能不写
System.out.println()括号里面可以写



一个类里面，可以写多个 public 的方法、多个 public 成员变量；但是【一个 `.java` 文件里最多只能有 1 个 public 类】

对！**一个方法执行完 return，它对应的栈帧就立刻释放。**

但是递归特殊点在这里：

> 外层方法**还没结束**，它在等着内层递归方法返回结果！

举 `factorial(3)`：

1. `fact(3)` 开始执行，调用 `fact(2)`
   → fact(3) **暂停，没有结束！** 栈帧保留，占内存

2. `fact(2)` 开始执行，调用 `fact(1)`
   → fact(2) **暂停，没有结束！** 栈帧保留，占内存

3. `fact(1)` 执行到出口 return 1
   → fact(1) **结束，栈帧释放** ✅

4. 回到 fact(2)，计算 `2*1`，return

→ fact(2) 结束，栈帧释放 ✅

5. 回到 fact(3)，计算 `3*2` ，return

→ fact(3) 结束，↓ 释放 ✅



表格

递归	迭代
自顶向下，将大问题分解为小问题（分治）。	自底向上，通过重复更新状态逐步逼近结果。
函数调用自身。	循环结构（for 、while 等）。
每次调用都需要栈空间，深度过大易导致**栈溢出**（Stack Overflow）。	通常只使用固定变量，内存消耗较小，不易溢出。
函数调用有额外开销（压栈、出栈），通常比迭代慢。	循环开销小，执行效率通常更高。
对于某些问题（如树遍历、分治算法），递归代码更简洁、直观。	对于简单重复任务，迭代更直接；复杂逻辑可能较繁琐。
必须有明确的**基准情况**，否则无限递归。	必须有明确的**循环终止条件**，否则无限循环。

为什么效率低

两个核心原因

1. **方法调用本身有开销**

   Java 调用方法时，JVM 要创建**栈帧**：保存当前方法的局部变量、返回地址等信息。

   每次递归调用都要压栈；方法返回还要出栈。

   大量的压栈、出栈操作，消耗时间。迭代没有反复的方法调用，只有变量计算。

2. **很多递归会产生大量重复计算（典型：斐波那契）**

   `fib(5)=fib(4)+fib(3)`

   算 fib (4) 的时候又要算一遍 fib (3)，fib (2)、fib (1) 被反复重复求解。

   随着 n 变大，计算量爆炸增长（时间复杂度 O (2^n)）。